

KUBERNETES (K8S) TASK - 08

1. Create and Test a Kubernetes Pod with an EmptyDir Volume

emptyDir Volume

Objective

Create a Pod with an emptyDir volume.

We will:

Pod

└─ nginx container

└─ emptyDir volume

└─ /usr/share/nginx/html

The volume exists as long as the **Pod exists**.

If the container restarts → data remains.

If the Pod is deleted → data is deleted.

Step 1: Check your cluster

Run from your **control-plane**:

```
kubectl get nodes
```

You should see something similar to:

NAME	STATUS	ROLES	AGE
k8s-master	Ready	control-plane	...
k8s-worker1	Ready	<none>	...
k8s-worker2	Ready	<none>	...

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get nodes  
NAME           STATUS    ROLES    AGE   VERSION  
k8s-master     Ready     control-plane   8d    v1.34.11  
k8s-worker1    Ready     <none>         8d    v1.34.11  
k8s-worker2    Ready     <none>         8d    v1.34.11  
ubuntu@k8s-master:~$ |
```

Step 2: Create YAML

nano emptydir-pod.yaml

```
GNU nano 8.7.1 emptydir-pod.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: emptydir-pod  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
    volumeMounts:  
    - name: shared-storage  
      mountPath: /usr/share/nginx/html  
  volumes:  
  - name: shared-storage  
    emptyDir: {}
```

Step 3: Create the Pod

kubectl apply -f emptydir-pod.yaml

Check:

kubectl get pod emptydir-pod -o wide

Expected:

emptydir-pod 1/1 Running

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f emptydir-pod.yaml  
pod/emptydir-pod created  
ubuntu@k8s-master:~$ kubectl get pod emptydir-pod -o wide  
NAME READY STATUS RESTARTS AGE IP NODE  
emptydir-pod 1/1 Running 0 9s 192.168.126.17 k8s-worker2  
<none> <none>  
ubuntu@k8s-master:~$ kubectl get pod emptydir-pod -o wide  
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES  
emptydir-pod 1/1 Running 0 24s 192.168.126.17 k8s-worker2 <none> <none>  
ubuntu@k8s-master:~$
```

Step 4: Create a file inside the volume

`kubectl exec -it emptydir-pod -- bash`

Inside the container:

`echo "Hello from emptyDir" > /usr/share/nginx/html/test.txt`

Check:

`cat /usr/share/nginx/html/test.txt`

Expected:

Hello from emptyDir

Exit:

`exit`

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl exec -it emptydir-pod -- bash  
root@emptydir-pod:/# echo "Hello from emptyDir" > /usr/share/nginx/html/test.txt  
root@emptydir-pod:/# cat /usr/share/nginx/html/test.txt  
Hello from emptyDir  
root@emptydir-pod:/# exit  
exit  
ubuntu@k8s-master:~$
```

Step 5: Verify the file

`kubectl exec emptydir-pod -- cat /usr/share/nginx/html/test.txt`

Expected:

Hello from emptyDir

Step 6: Test container restart

We can delete the container indirectly by deleting the Pod only to demonstrate the persistence rule.

First, remember:

container restart → emptyDir survives

Pod deletion → emptyDir is deleted

To demonstrate Pod deletion:

```
kubectl delete pod emptydir-pod
```

Create it again:

```
kubectl apply -f emptydir-pod.yaml
```

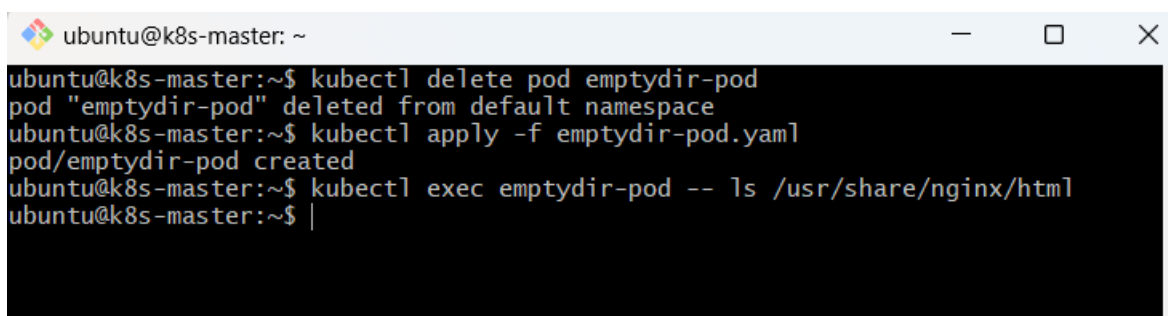
Check:

```
kubectl exec emptydir-pod -- ls /usr/share/nginx/html
```

test.txt will be gone.

Important interview point

emptyDir provides temporary storage shared between containers in the same Pod. Its data survives container restarts but is deleted when the Pod is removed.

A terminal window titled 'ubuntu@k8s-master: ~' with standard window controls. The terminal shows the following commands and output:

```
ubuntu@k8s-master:~$ kubectl delete pod emptydir-pod
pod "emptydir-pod" deleted from default namespace
ubuntu@k8s-master:~$ kubectl apply -f emptydir-pod.yaml
pod/emptydir-pod created
ubuntu@k8s-master:~$ kubectl exec emptydir-pod -- ls /usr/share/nginx/html
ubuntu@k8s-master:~$ |
```

2. Configure a HostPath Volume in Kubernetes and Validate Data Persistence

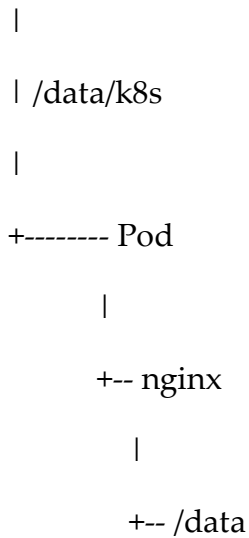
HostPath Volume

Objective

We will store data directly on the **worker node's filesystem**.

Architecture:

Worker Node



Step 1: Find where the Pod is running

kubectl get pod emptydir-pod -o wide

For example:

NAME	NODE
emptydir-pod	k8s-worker1

We will create our hostPath directory on that worker.

Step 1: Find where the Pod is running

kubectl get pod emptydir-pod -o wide

For example:

NAME	NODE
emptydir-pod	k8s-worker1

We will create our hostPath directory on that worker.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get pod emptydir-pod -o wide  
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE  
NOMINATED NODE READINESS GATES  
emptydir-pod  1/1     Running   0           54m   192.168.126.16 k8s-worker2  
<none>         <none>  
ubuntu@k8s-master:~$ |
```

Step 2: SSH into worker2

From your Windows terminal:

```
ssh -i docker.pem ubuntu@<worker1-public-ip>
```

Then:

```
sudo mkdir -p /data/k8s
```

Give permission:

```
sudo chmod 777 /data/k8s
```

Create a test file:

```
echo "Data from worker node" | sudo tee /data/k8s/test.txt
```

Check:

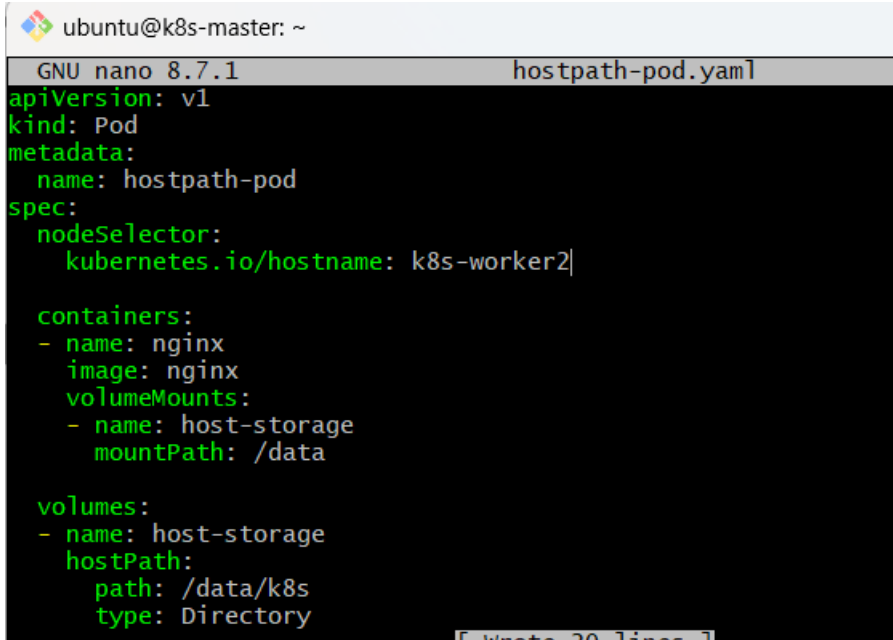
```
cat /data/k8s/test.txt
```

```
us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addres...  
aws [Search] [Alt+S] United States (N. Virginia) badugu (820982384063) badugu  
ubuntu@k8s-worker2:~$ sudo mkdir -p /data/k8s  
ubuntu@k8s-worker2:~$ sudo chmod 777 /data/k8s  
ubuntu@k8s-worker2:~$ echo "Data from worker2 node from Rajkumari" | ^Cdo tee /data/k8s/test.txt  
ubuntu@k8s-worker2:~$ ^C  
ubuntu@k8s-worker2:~$ ^C  
ubuntu@k8s-worker2:~$ echo "Data from worker node from Rajkumari " | sudo tee /data/k8s/test.txt  
Data from worker node from Rajkumari  
ubuntu@k8s-worker2:~$ cat /data/k8s/test.txt  
Data from worker node from Rajkumari  
ubuntu@k8s-worker2:~$ |
```

Step 3: Create hostPath Pod

Go back to the control-plane:

nano hostpath-pod.yaml

A screenshot of a terminal window. The top bar shows 'ubuntu@k8s-master: ~'. Below it, the nano editor header shows 'GNU nano 8.7.1' and the filename 'hostpath-pod.yaml'. The main content is a YAML file for a Pod. The text is as follows:

```
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-pod
spec:
  nodeSelector:
    kubernetes.io/hostname: k8s-worker2
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: host-storage
      mountPath: /data
  volumes:
  - name: host-storage
    hostPath:
      path: /data/k8s
      type: Directory
```

The important part is:

nodeSelector:

kubernetes.io/hostname: k8s-worker2

Why?

Because /data/k8s exists on **worker2**

If Kubernetes moves the Pod to worker2 and /data/k8s doesn't exist there, the Pod can fail.

Step 4: Create Pod

kubectl apply -f hostpath-pod.yaml

Check:

kubectl get pod hostpath-pod -o wide

It should be on:

k8s-worker2

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f hostpath-pod.yaml  
pod/hostpath-pod created  
ubuntu@k8s-master:~$ kubectl get pod hostpath-pod -o wide  
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE  
NOMINATED     NODE    READINESS GATES  
hostpath-pod  1/1     Running   0           7s    192.168.126.21 k8s-worker2  
<none>        <none>  
ubuntu@k8s-master:~$ |
```

Step 5: Verify the host file from Pod

`kubectl exec hostpath-pod -- cat /data/test.txt`

Expected:

Data from worker node

This proves:

Worker:

`/data/k8s/test.txt`

↓

Pod:

`/data/test.txt`

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl exec hostpath-pod -- cat /data/test.txt  
Data from worker node from Rajkumari  
ubuntu@k8s-master:~$ |
```

Step 6: Write data from Pod

`kubectl exec hostpath-pod -- sh -c 'echo "Written from Kubernetes Pod" >`

`/data/pod.txt'`

Now SSH to worker1:

```
cat /data/k8s/pod.txt
```

Expected:

Written from Kubernetes Pod

Important interview point

hostPath is tied to a **specific node's filesystem**.

Therefore it is generally not the preferred solution for production application persistence across multiple nodes.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl exec hostpath-pod -- sh -c 'echo "Written from Kube  
rnetes Pod" > /data/pod.txt'  
ubuntu@k8s-master:~$ kubectl exec hostpath-pod -- sh -c 'echo "Written from Kube  
rnetes Pod" > /data/pod.txt'  
ubuntu@k8s-master:~$ kubectl exec hostpath-pod -- sh -c 'echo "Written from Kubernetes Pod" > /data/pod.txt'  
ubuntu@k8s-master:~$ |
```

```
aws | [Search] [Alt+S] | United States (N. Virginia) | badugu (820982384063) | badugu  
ubuntu@k8s-worker2:~$ sudo mkdir -p /data/k8s  
ubuntu@k8s-worker2:~$ sudo chmod 777 /data/k8s  
ubuntu@k8s-worker2:~$ echo "Data from worker2 node from Rajkumari" | ^Cdo tee /data/k8s/test.txt  
ubuntu@k8s-worker2:~$ ^C  
ubuntu@k8s-worker2:~$ echo "Data from worker node from Rajkumari " | sudo tee /data/k8s/test.txt  
Data from worker node from Rajkumari  
ubuntu@k8s-worker2:~$ cat /data/k8s/test.txt  
Data from worker node from Rajkumari  
ubuntu@k8s-worker2:~$ cat /data/k8s/pod.txt  
Written from Kubernetes Pod  
ubuntu@k8s-worker2:~$ |
```

3. Deploy an Amazon EBS Volume Using Persistent Volume and Persistent Volume Claim (PVC)

Amazon EBS + PV + PVC

Now we move to **persistent storage**.

Architecture:

Kubernetes Pod

|

↓

PVC

|

↓

PV

|

↓

Amazon EBS

|

↓

AWS Availability Zone

Step 3.1 — Understand the components

PersistentVolume — PV

A PV represents storage available to Kubernetes.

Example:

PV = 10 GiB EBS storage

PersistentVolumeClaim — PVC

PVC is the application's request for storage.

Example:

"I need 5 GiB storage"

Pod

The Pod uses the PVC.

Pod → PVC → PV → EBS

Step 3.2 — Check EBS CSI Driver

Run:

```
kubectl get pods -n kube-system | grep ebs
```

You should see something similar to:

```
ebs-csi-controller-xxxxx
```

```
ebs-csi-node-xxxxx
```

Also:

```
kubectl get csidriver
```

Look for:

```
ebs.csi.aws.com
```

If you don't have the EBS CSI driver, **don't continue with the EBS YAML yet.**

Create volume [Info](#)

Create an Amazon EBS volume to attach to any EC2 instance in the same Availability Zone.

Volume settings

Volume type [Info](#)

General Purpose SSD (gp3)

Size (GiB) [Info](#)

100

Min: 1 GiB, Max: 65536 GiB.

IOPS [Info](#)

3000

Min: 3000 IOPS, Max: 80000 IOPS.

Throughput (MiB/s) [Info](#)

125

Min: 125 MiB, Max: 2000 MiB, Baseline: 125 MiB/s.

Availability Zone [Info](#)

us-east-1a

Snapshot ID - optional [Info](#)

Don't create volume from a snapshot

Encryption [Info](#)

Use Amazon EBS encryption as an encryption solution for your EBS resources associated with your EC2 instances.

☐ Encrypt this volume

Volumes (6) [Info](#)

Successfully created volume [vol-06ca3599ed45b27f2](#).

Last updated less than a minute ago [Recycle Bin](#) [Actions](#) [Create volume](#)

Type	Size	IOPS	Throughput	Owner ID	Snapshot ID	Source volume ID	Created	Availability Zone	Volume state	Alarm status
gp3	100 GiB	3000	125	820982384063	-	-	2026/09/30 21:09 GMT+5...	use1-az1 (us-east-1a)	Available	No alarms
gp3	8 GiB	3000	125	820982384063	snap-0ffe259e...	-	2026/09/22 13:48 GMT+5...	use1-az4 (us-east-1c)	In-use	No alarms
gp3	8 GiB	3000	125	820982384063	snap-0ffe259e...	-	2026/09/22 13:48 GMT+5...	use1-az4 (us-east-1c)	In-use	No alarms
gp3	8 GiB	3000	125	820982384063	snap-0fc7915...	-	2026/09/07 17:42 GMT+5...	use1-az6 (us-east-1d)	In-use	No alarms
gp3	8 GiB	3000	125	820982384063	snap-0fc7915...	-	2026/09/01 12:51 GMT+5...	use1-az4 (us-east-1c)	In-use	No alarms
gp3	8 GiB	3000	125	820982384063	snap-0ffe259e...	-	2026/09/22 13:48 GMT+5...	use1-az4 (us-east-1c)	In-use	No alarms

Step 1 Select trusted entity
Step 2 **Add permissions**
Step 3 Name, review, and create

Add permissions info

☒ Use existing policy Attach an existing managed policy to the role.
☐ Create inline policy Create a new policy that will be attached directly to the role.

Permissions policies (1/1221) info
Choose one or more policies to attach to your new role.

Filter by Type: All types 4 matches

Policy name info	Type	Description
☐ AmazonEBS	AWS managed	
☒ AmazonEBSDriverPolicy	AWS managed	
☐ AmazonEBSDriverPolicyV2	AWS managed	
☐ ROSAAmazonEBSDriverOperatorPolicy	AWS managed	

► Set permissions boundary - optional

Cancel Previous Next

Identity and Access Management (IAM)

Dashboard

▼ Access Management

- Roles**
- Policies
- IAM users
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

▼ Access reports

- Access Analyzer
- Resource analysis
- Unused access
- Analyzer settings
- Policy simulator
- Credential report

▼ Multi-account access

- Account access manager
- Organization activity
- Service control policies
- Resource control policies

Role ebs-role created.

[View role](#)

Roles (7) info

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

Role name	Trusted entities	Last activity
AWSServiceRoleForElasticLoadBalancing	AWS Service: elasticloadbalancing (Service-Linked Role)	10 days ago
AWSServiceRoleForResourceExplorer	AWS Service: resource-explorer-2 (Service-Linked Role)	5 minutes ago
AWSServiceRoleForSupport	AWS Service: support (Service-Linked Role)	-
AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service-Linked Role)	-
☒ ebs-role	AWS Service: ec2	-
terraform	AWS Service: ec2	23 days ago
Terraform-jenkins	AWS Service: ec2	26 days ago

Roles Anywhere info

Authenticate your non AWS workloads and securely provide access to AWS services.

Access AWS from your non AWS workloads
Operate your non AWS workloads using the same authentication and authorization strategy that you use within AWS.

X.509 Standard
Use your own existing PKI infrastructure or use [AWS Certificate Manager Private Certificate Authority](#) to authenticate identities.

Temporary credentials
Use temporary credentials with ease and benefit from the enhanced security they provide.

[Manage](#)

EC2 > [Instances](#) > [i-01b64330d55a2dc07](#) > Modify IAM role

Modify IAM role info

Attach an IAM role to your instance.

Instance ID: [i-01b64330d55a2dc07](#) (k8s-master)

IAM role: Select an IAM role to attach to your instance or create a new role if you haven't created any. The role you select replaces any roles that are currently attached to your instance.

[ebs-role](#) [Create new IAM role](#)

Cancel Update IAM role

EC2 > [Instances](#) > [i-09225ab91ef587553](#) > Modify IAM role

Modify IAM role info

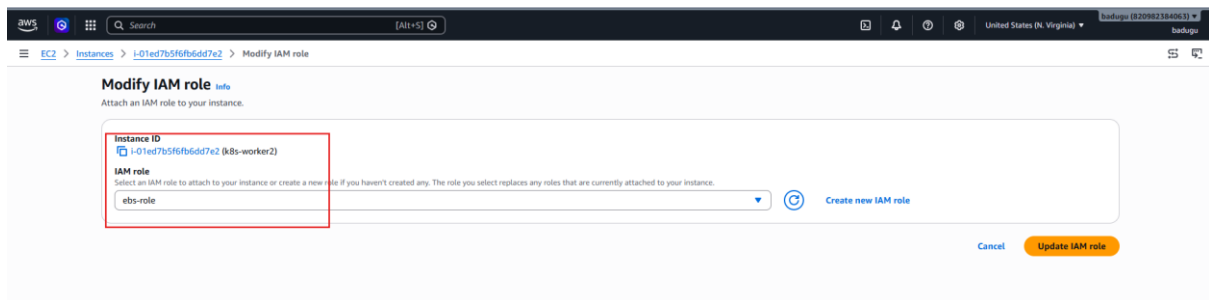
Attach an IAM role to your instance.

Instance ID: [i-09225ab91ef587553](#) (k8s-worker1)

IAM role: Select an IAM role to attach to your instance or create a new role if you haven't created any. The role you select replaces any roles that are currently attached to your instance.

[ebs-role](#) [Create new IAM role](#)

Cancel Update IAM role



```
ubuntu@k8s-master: ~/aws-ebs-csi-driver
ubuntu@k8s-master:~$ git clone https://github.com/kubernetes-sigs/aws-ebs-csi-driver.git
Cloning into 'aws-ebs-csi-driver'...
remote: Enumerating objects: 38869, done.
remote: Counting objects: 100% (456/456), done.
remote: Compressing objects: 100% (193/193), done.
remote: Total 38869 (delta 358), reused 271 (delta 262), pack-reused 38413 (from 3)
Receiving objects: 100% (38869/38869), 31.31 MiB | 35.23 MiB/s, done.
Resolving deltas: 100% (22836/22836), done.
ubuntu@k8s-master:~$ cd aws-ebs-csi-driver
ubuntu@k8s-master:~/aws-ebs-csi-driver$ kubectl apply -k deploy/kubernetes/overl
aws/stable/
serviceaccount/ebs-csi-controller-sa created
serviceaccount/ebs-csi-node-sa created
role.rbac.authorization.k8s.io/ebs-csi-leases-role created
clusterrole.rbac.authorization.k8s.io/ebs-csi-node-role created
clusterrole.rbac.authorization.k8s.io/ebs-external-attacher-role created
clusterrole.rbac.authorization.k8s.io/ebs-external-provisioner-role created
clusterrole.rbac.authorization.k8s.io/ebs-external-resizer-role created
clusterrole.rbac.authorization.k8s.io/ebs-external-snapshotter-role created
rolebinding.rbac.authorization.k8s.io/ebs-csi-leases-rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/ebs-csi-attacher-binding created
clusterrolebinding.rbac.authorization.k8s.io/ebs-csi-node-getter-binding created
clusterrolebinding.rbac.authorization.k8s.io/ebs-csi-provisioner-binding created
clusterrolebinding.rbac.authorization.k8s.io/ebs-csi-resizer-binding created
clusterrolebinding.rbac.authorization.k8s.io/ebs-csi-snapshotter-binding created
deployment.apps/ebs-csi-controller created
poddisruptionbudget.policy/ebs-csi-controller created
daemonset.apps/ebs-csi-node created
csidriver.storage.k8s.io/ebs.csi.aws.com created
ubuntu@k8s-master:~/aws-ebs-csi-driver$ kubectl get pods -n kube-system
NAME                                READY    STATUS    RESTARTS   AGE
calico-kube-controllers-f4cf54b75-bhgjr 0/1      CrashLoopBackOff    262 (3m29s ago)    8d
calico-node-7bqzp                      1/1      Running    8 (112m ago)    8d
calico-node-bpw4p                      1/1      Running    8 (112m ago)    8d
calico-node-hgltg                      1/1      Running    8 (112m ago)    8d
coredns-66bc5c9577-s4rsp              1/1      Running    8 (112m ago)    8d
coredns-66bc5c9577-sdmpt              1/1      Running    8 (112m ago)    8d
ebs-csi-controller-5c47f7d4d6-h992v    6/6      Running    0            28s
ebs-csi-controller-5c47f7d4d6-htp2d    6/6      Running    0            28s
ebs-csi-node-5g52g                    3/3      Running    0            28s
ebs-csi-node-8f9gr                    3/3      Running    0            28s
ebs-csi-node-ld6hz                    3/3      Running    0            28s
etcd-k8s-master                       1/1      Running    8 (112m ago)    8d
kube-apiserver-k8s-master              1/1      Running    8 (112m ago)    8d
kube-controller-manager-k8s-master    1/1      Running    8 (112m ago)    8d
kube-proxy-cwmmc                      1/1      Running    8 (112m ago)    8d
kube-proxy-h8fls                      1/1      Running    8 (112m ago)    8d
kube-proxy-mqsvs                      1/1      Running    8 (112m ago)    8d
kube-scheduler-k8s-master              1/1      Running    8 (112m ago)    8d
metrics-server-7c5fd4664-sxhtq        1/1      Running    2 (111m ago)    24h
ubuntu@k8s-master:~/aws-ebs-csi-driver$
```

```
ubuntu@k8s-master: ~
ubuntu@k8s-master:~/aws-ebs-csi-driver$ cd ..
ubuntu@k8s-master:~$ kubectl get pods -n kube-system | grep ebs
ebs-csi-controller-5c47f7d4d6-h992v    6/6      Running    0            2m
41s
ebs-csi-controller-5c47f7d4d6-htp2d    6/6      Running    0            2m
41s
ebs-csi-node-5g52g                    3/3      Running    0            2m
41s
ebs-csi-node-8f9gr                    3/3      Running    0            2m
41s
ebs-csi-node-ld6hz                    3/3      Running    0            2m
41s
ubuntu@k8s-master:~$ kubectl get csidriver
NAME                                ATTACHREQUIRED  PODINFOONMOUNT  STORAGECAPACITY  TOKENREQUESTS  REQUIRESREPUBLISH  MODES  AGE
ebs.csi.aws.com  true            false           false             <unset>         false              Persistent  3m33s
ubuntu@k8s-master:~$
```

Step 3.3 — Check AWS permissions

Your EC2 worker nodes need permissions to use EBS through the CSI driver.

From the worker:

```
aws sts get-caller-identity
```

If AWS CLI isn't installed:

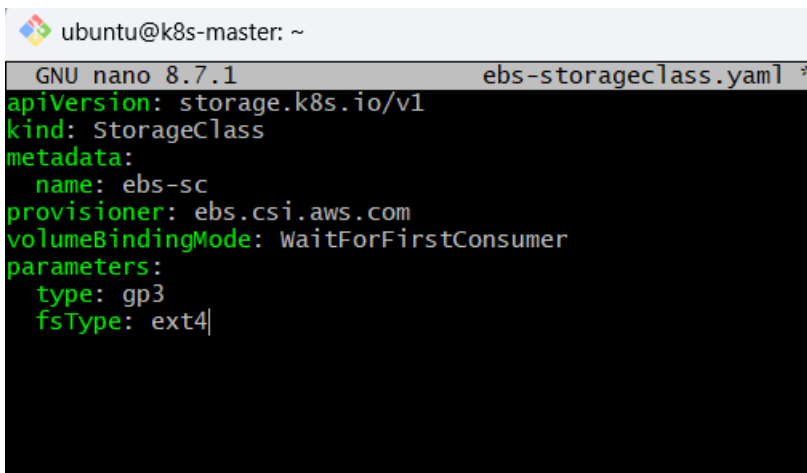
```
aws --version
```

The better production approach is to use an IAM role/service account rather than hard-coded AWS access keys.

Step 3.4 — Create StorageClass

Create:

Nano ebs-storageclass.yaml



```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 ebs-storageclass.yaml *  
apiVersion: storage.k8s.io/v1  
kind: StorageClass  
metadata:  
  name: ebs-sc  
provisioner: ebs.csi.aws.com  
volumeBindingMode: WaitForFirstConsumer  
parameters:  
  type: gp3  
  fsType: ext4
```

Apply:

```
kubectl apply -f ebs-storageclass.yaml
```

Check:

```
kubectl get storageclass
```

Expected:

NAME	PROVISIONER
------	-------------

ebs-sc	ebs.csi.aws.com
--------	-----------------

Why WaitForFirstConsumer?

EBS volumes are **Availability Zone specific**.

This setting allows Kubernetes to consider where the Pod will run before dynamically creating the EBS volume.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano ebs-storageclass.yaml  
ubuntu@k8s-master:~$ kubectl apply -f ebs-storageclass.yaml  
storageclass.storage.k8s.io/ebs-sc created  
ubuntu@k8s-master:~$ kubectl get storageclass  
NAME          PROVISIONER          RECLAIMPOLICY    VOLUMEBINDINGMODE    ALLOWVOLUMEEXP  
ANSION        AGE  
ebs-sc        ebs.csi.aws.com      Delete           WaitForFirstConsumer  false  
              7s  
ubuntu@k8s-master:~$ |
```

Step 3.5 — Create PVC

nano ebs-pvc.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 ebs-pvc.yaml *  
apiVersion: v1  
kind: PersistentVolumeClaim  
metadata:  
  name: ebs-pvc  
spec:  
  accessModes:  
    - ReadWriteOnce  
  storageClassName: ebs-sc  
  resources:  
    requests:  
      storage: 5Gi|
```

Apply:

kubectl apply -f ebs-pvc.yaml

Check:

kubectl get pvc

Initially it may show:

NAME	STATUS	VOLUME
------	--------	--------

ebs-pvc	Pending	
---------	---------	--

That can be normal because we used:

WaitForFirstConsumer

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f ebs-pvc.yaml  
persistentvolumeclaim/ebs-pvc created  
ubuntu@k8s-master:~$ kubectl get pvc  
NAME          STATUS      VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTR  
ebs-pvc       Pending    12s  
ebs-sc        <unset>
```

Step 3.6 – Create Pod

Nano ebs-pod.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: v1  
kind: Pod  
metadata:  
  name: ebs-pod  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
    command: ["/bin/sh", "-c"]  
    args:  
    - |  
      echo "Hello from EBS volume" > /data/test.txt  
      sleep 3600  
  volumeMounts:  
  - name: ebs-storage  
    mountPath: /data  
  
volumes:  
  - name: ebs-storage  
    persistentVolumeClaim:  
      claimName: ebs-pvc
```

Create:

```
kubectl apply -f ebs-pod.yaml
```

Check:

```
kubectl get pod ebs-pod -o wide
```

Then:

kubectl get pvc

Expected:

NAME	STATUS	VOLUME
ebs-pvc	Bound	pvc-xxxxxxxxx

Check PV:

kubectl get pv

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano ebs-pod.yaml  
ubuntu@k8s-master:~$ kubectl apply -f ebs-pod.yaml  
pod/ebs-pod created  
ubuntu@k8s-master:~$ kubectl get pod ebs-pod -o wide  
NAME READY STATUS RESTARTS AGE IP NODE NO  
MINATED NODE READINESS GATES  
ebs-pod 0/1 ContainerCreating 0 6s <none> k8s-worker2 <n  
one>  
ubuntu@k8s-master:~$ kubectl get pvc  
NAME STATUS VOLUME CAPACITY ACCESS  
MODES STORAGECLASS VOLUMEATTRIBUTESCLASS AGE  
ebs-pvc Bound pvc-0bd1d079-e839-4cb6-ba68-f22206c41dcb 5Gi RWO  
ebs-sc <unset> 2m36s  
ubuntu@k8s-master:~$ kubectl get pod ebs-pod -o wide  
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES  
ebs-pod 0/1 Completed 0 29s 192.168.126.19 k8s-worker2 <none> <none>  
ubuntu@k8s-master:~$ kubectl get pvc  
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCLASS AGE  
ebs-pvc Bound pvc-0bd1d079-e839-4cb6-ba68-f22206c41dcb 5Gi RWO ebs-sc <unset> 2m57s  
ubuntu@k8s-master:~$ kubectl get pv  
NAME CAPACITY ACCESS MODES RECLAIM POLICY STATUS CLAIM default/ebs-pvc STORAGECLASS VOLUMEATTRIBUTESCLASS REASON AGE  
pvc-0bd1d079-e839-4cb6-ba68-f22206c41dcb 5Gi RWO Delete Bound default/ebs-pvc ebs-sc <unset> 43s  
ubuntu@k8s-master:~$
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f ebs-pod.yaml  
pod/ebs-pod created  
ubuntu@k8s-master:~$ kubectl get pod ebs-pod -o wide  
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GAT  
ebs-pod 1/1 Running 0 10s 192.168.194.96 k8s-worker1 <none> <none>  
ubuntu@k8s-master:~$
```

Step 3.7 — Write data

kubectl exec -it ebs-pod -- bash

Inside:

echo "Important persistent data" > /data/database.txt

Verify:

cat /data/database.txt

Exit:

Exit

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl exec -it ebs-pod -- bash  
root@ebs-pod:/# echo "Important persistent data" > /data/database.txt  
root@ebs-pod:/# cat /data/database.txt  
Important persistent data  
root@ebs-pod:/# ll  
bash: ll: command not found  
root@ebs-pod:/# ls  
bin  data  docker-entrypoint.d  etc  lib  media  opt  root  sbin  sys  usr  
boot dev  docker-entrypoint.sh  home  lib64  mnt  proc  run  srv  tmp  var  
root@ebs-pod:/# exit  
exit  
ubuntu@k8s-master:~$ |
```

Step 3.8 — Delete Pod

`kubectl delete pod ebs-pod`

Recreate:

`kubectl apply -f ebs-pod.yaml`

Check:

`kubectl exec ebs-pod -- cat /data/database.txt`

Expected:

Important persistent data

This demonstrates **persistent storage**.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl delete pod ebs-pod  
pod "ebs-pod" deleted from default namespace  
ubuntu@k8s-master:~$ kubectl apply -f ebs-pod.yaml  
pod/ebs-pod created  
ubuntu@k8s-master:~$ kubectl exec ebs-pod -- cat /data/database.txt  
Important persistent data  
ubuntu@k8s-master:~$ |
```

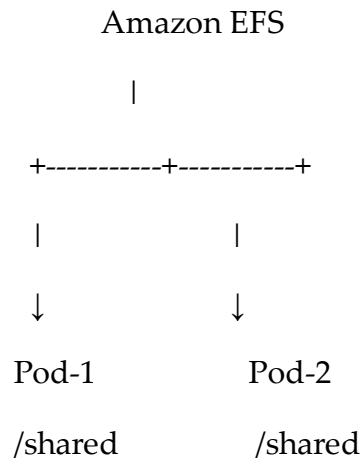
4.Set Up an Amazon EFS Volume and Attach it to Multiple Pods

Amazon EFS + Multiple Pods

This is an important real-world scenario.

Unlike EBS, EFS can be mounted by **multiple Pods simultaneously**.

Architecture:



EFS supports:

ReadWriteMany

Step 4.1 — Check EFS CSI Driver

Run:

```
kubectl get pods -n kube-system | grep efs
```

You should see:

```
efs-csi-controller-xxxxx
```

```
efs-csi-node-xxxxx
```

Check:

```
kubectl get csidriver
```

Look for:

```
efs.csi.aws.com
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get pods -n kube-system | grep efs  
ubuntu@k8s-master:~$ kubectl get csidriver  
NAME          AGE          ATTACHREQUIRED  PODINFOONMOUNT  STORAGECAPACITY  TOKENREQUESTS  REQUIRESREUBLISH  
ebs.csi.aws.com  true         false           false           false             <unset>         false  
ent 121m  
ubuntu@k8s-master:~$ |
```

Step 4.2 — Create EFS filesystem

Go to AWS Console:

AWS Console



EFS



Create file system

Create an EFS filesystem in the **same VPC** as your Kubernetes nodes.

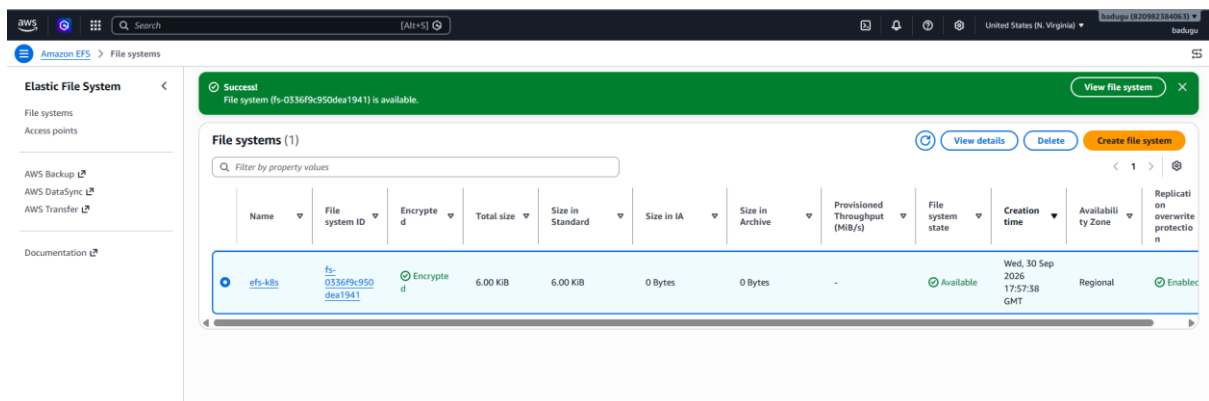
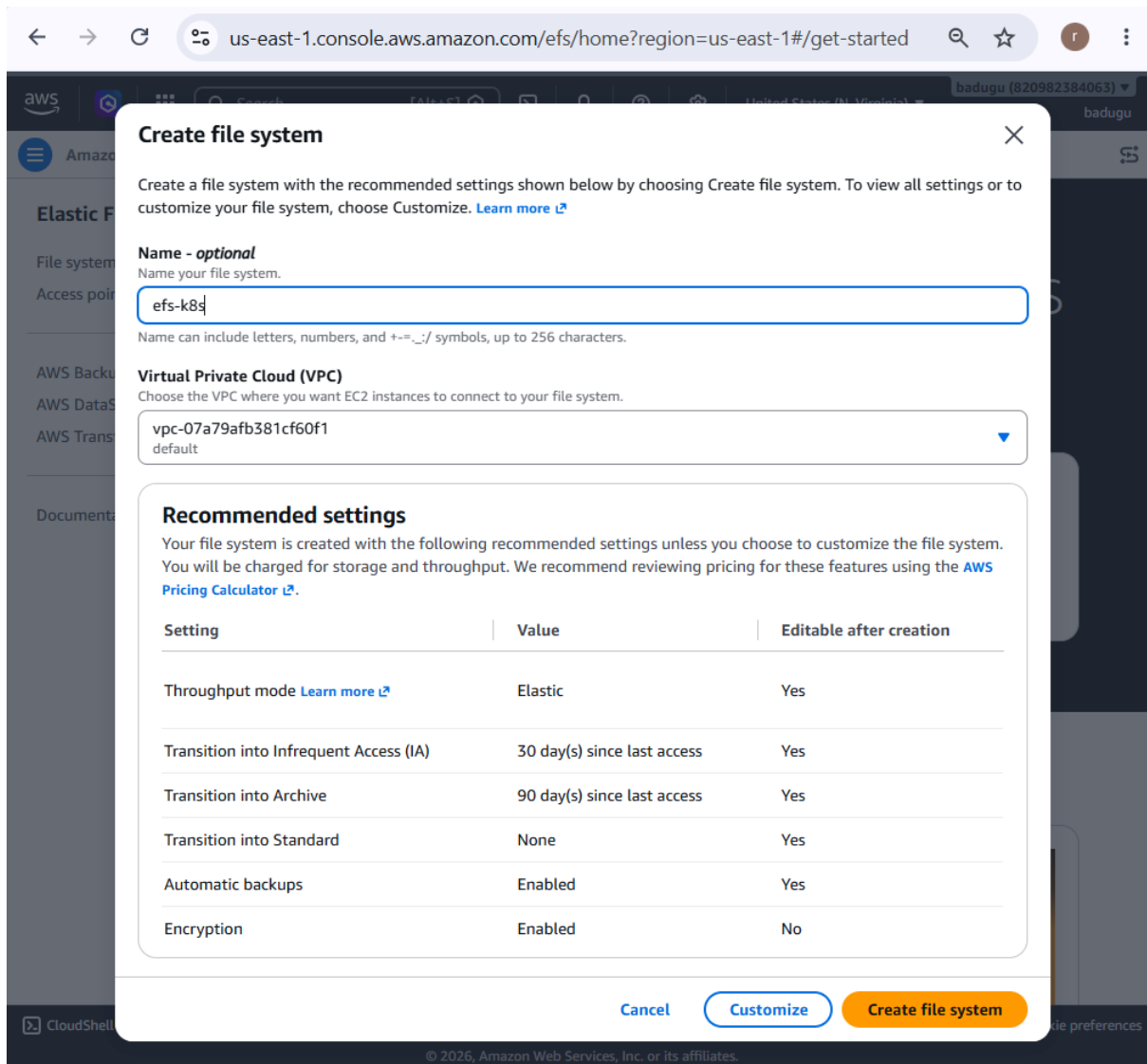
Make sure the EFS filesystem has mount targets accessible from the worker-node subnets.

Record the filesystem ID:

fs-xxxxxxxxxxxxxxxxxxx

For example:

fs-0123456789abcdef



Step 4.3 — Create EFS PV

On the Kubernetes master:

```
nano efs-pv.yaml
```

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 efs-pv.yaml  
apiVersion: v1  
kind: PersistentVolume  
metadata:  
  name: efs-pv  
spec:  
  capacity:  
    storage: 5Gi  
  
  volumeMode: Filesystem  
  
  accessModes:  
    - ReadWriteMany  
  
  persistentVolumeReclaimPolicy: Retain  
  
  csi:  
    driver: efs.csi.aws.com  
    volumeHandle: fs-0336f9c950dea1941
```

Replace:

fs-xxxxxxxxxxxxxxxxxxxx

with your actual EFS filesystem ID.

Apply:

kubectl apply -f efs-pv.yaml

Check:

kubectl get pv

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano efs-pv.yaml  
ubuntu@k8s-master:~$ kubectl apply -f efs-pv.yaml  
persistentvolume/efs-pv created  
ubuntu@k8s-master:~$ kubectl get pv  
NAME                                CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM                STORAGECLASS  VOLUMEATTRIBUTESCLASS  REASON  AGE  
efs-pv                              5Gi       RWX           Retain          Available                                ebs-sc        <unset>              15s  
pvc-0bd1d079-e839-4cb6-ba68-f22206c41dcb  5Gi       RWO           Delete          Bound    default/ebs-pvc    ebs-sc        <unset>              116m  
ubuntu@k8s-master:~$ kubectl get pv  
NAME                                CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM                STORAGECLASS  VOLUMEATTRIBUTESCLASS  REASON  AGE  
efs-pv                              5Gi       RWX           Retain          Available                                ebs-sc        <unset>              15s  
pvc-0bd1d079-e839-4cb6-ba68-f22206c41dcb  5Gi       RWO           Delete          Bound    default/ebs-pvc    ebs-sc        <unset>              116m  
ubuntu@k8s-master:~$
```

Step 4.4 — Create PVC

nano efs-pvc.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: v1  
kind: PersistentVolumeClaim  
metadata:  
  name: efs-pvc  
spec:  
  accessModes:  
    - ReadWriteMany  
  
  resources:  
    requests:  
      storage: 5Gi  
  
  volumeName: efs-pv|
```

Apply:

```
kubectl apply -f efs-pvc.yaml
```

Check:

```
kubectl get pvc
```

Expected:

```
efs-pvc  Bound
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano efs-pvc.yaml  
ubuntu@k8s-master:~$ kubectl apply -f efs-pvc.yaml  
persistentvolumeclaim/efs-pvc created  
ubuntu@k8s-master:~$ kubectl get pvc  
NAME          STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTR  
ebs-pvc       Bound     pvc-0bd1d079-e839-4cb6-ba68-f22206c41dcb  5Gi        RWO            ebs-sc         <unset>  
efs-pvc       Bound     efs-pv                                     5Gi        RWX            <unset>  
ubuntu@k8s-master:~$
```

Step 4.5 — Create Pod 1

```
nano efs-pod1.yaml
```

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: v1  
kind: Pod  
metadata:  
  name: efs-pod1  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
  
  volumeMounts:  
  - name: efs-storage  
    mountPath: /shared  
  
  volumes:  
  - name: efs-storage  
    persistentVolumeClaim:  
      claimName: efs-pvc
```

Step 4.6 — Create Pod 2

Nano efs-pod2.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: v1  
kind: Pod  
metadata:  
  name: efs-pod2  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
  
  volumeMounts:  
  - name: efs-storage  
    mountPath: /shared  
  
  volumes:  
  - name: efs-storage  
    persistentVolumeClaim:  
      claimName: efs-pvc
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano efs-pod1.yaml  
ubuntu@k8s-master:~$ kubectl apply -f efs-pod1.yaml  
pod/efs-pod1 created  
ubuntu@k8s-master:~$ nano efs-pod2.yaml  
ubuntu@k8s-master:~$ kubectl apply -f efs-pod2.yaml  
pod/efs-pod2 created  
ubuntu@k8s-master:~$ kubectl get pods -o wide  
NAME          READY   STATUS             RESTARTS   AGE   IP            NODE          NOMINATED NODE   READINESS GATES  
efs-pod1     0/1     ContainerCreating   0          7m   <none>        k8s-worker1   <none>           <none>  
efs-pod2     0/1     ContainerCreating   0          7s   <none>        k8s-worker1   <none>           <none>
```

Step 4.7 — Write from Pod 1

```
kubectl exec efs-pod1 -- sh -c 'echo "Hello from Pod 1" > /shared/shared.txt'
```

Step 4.8 — Read from Pod 2

```
kubectl exec efs-pod2 -- cat /shared/shared.txt
```

Expected:

Hello from Pod 1

This proves both Pods are using the same EFS filesystem.

EBS vs EFS

Remember this for interviews:

Feature	EBS	EFS
Type	Block storage	File storage
AWS service	Elastic Block Store	Elastic File System
Multiple Pods	Usually not simultaneously	Yes
Access mode	RWO commonly	RWX
Shared storage	No	Yes
Availability Zone	AZ-specific	Regional
Kubernetes CSI	EBS CSI	EFS CSI
Typical use	Database / application disk	Shared files

5. Implement and Test Liveness and Readiness Probes in a Kubernetes Pod

Liveness and Readiness Probes

Now we'll test Kubernetes application health.

There are two important concepts:

Liveness Probe

Answers:

"Is the application still alive?"

If the liveness probe fails repeatedly, Kubernetes can **restart the container**.

Readiness Probe

Answers:

"Is the application ready to receive traffic?"

If readiness fails:

Pod remains running

↓

but removed from Service endpoints

Step 5.1 — Create Probe Pod

Nano probe-pod.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: v1  
kind: Pod  
metadata:  
  name: probe-pod  
  labels:  
    app: probe-demo  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
  
  ports:  
  - containerPort: 80  
  
  livenessProbe:  
    httpGet:  
      path: /  
      port: 80  
    initialDelaySeconds: 10  
    periodSeconds: 5  
  
  readinessProbe:  
    httpGet:  
      path: /  
      port: 80  
    initialDelaySeconds: 5  
    periodSeconds: 5
```

Create:

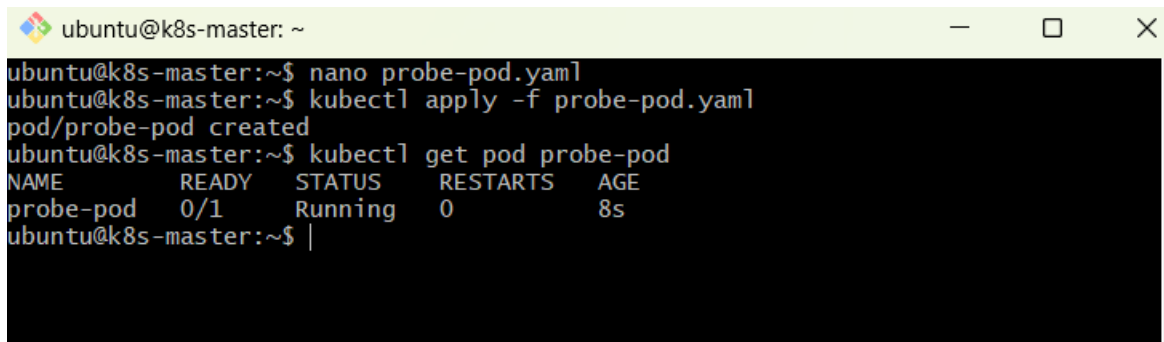
```
kubectl apply -f probe-pod.yaml
```

Step 5.2 — Check Pod

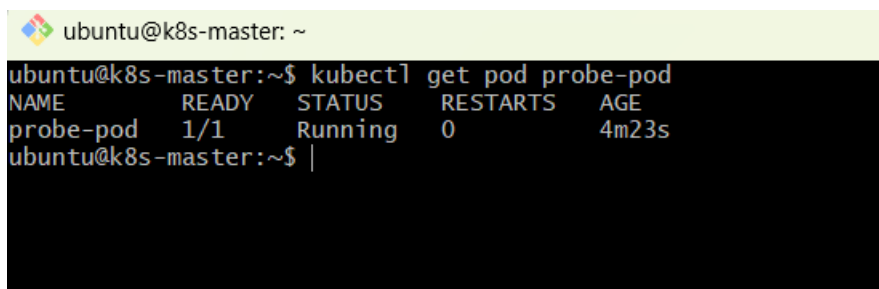
```
kubectl get pod probe-pod
```

Expected:

```
probe-pod 1/1 Running
```



```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano probe-pod.yaml  
ubuntu@k8s-master:~$ kubectl apply -f probe-pod.yaml  
pod/probe-pod created  
ubuntu@k8s-master:~$ kubectl get pod probe-pod  
NAME      READY   STATUS    RESTARTS   AGE  
probe-pod 0/1     Running   0           8s  
ubuntu@k8s-master:~$ |
```



```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get pod probe-pod  
NAME      READY   STATUS    RESTARTS   AGE  
probe-pod 1/1     Running   0           4m23s  
ubuntu@k8s-master:~$ |
```

Step 5.3 — Check probe information

```
kubectl describe pod probe-pod
```

Scroll down to:

Liveness

Readiness

You should see something similar to:

Liveness: http-get http://:80/ delay=10s timeout=1s period=5s

Readiness: http-get http://:80/ delay=5s timeout=1s period=5s

```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl describe pod probe-pod
Name: probe-pod
Namespace: default
Priority: 0
Service Account: default
Node: k8s-worker2/172.31.26.51
Start Time: Thu, 01 Oct 2026 08:17:11 +0000
Labels: app=probe-demo
Annotations: cni.projectcalico.org/containerID: 2e36c9cec541891ad48f781dda7c47206892b51fe8458322f1fc16aadf2d098c
            cni.projectcalico.org/podIP: 192.168.126.44/32
            cni.projectcalico.org/podIPs: 192.168.126.44/32
Status: Running
IP: 192.168.126.44
IPs:
  IP: 192.168.126.44
Containers:
  nginx:
    Container ID: containerd://d91e92e24dc983a7af29b636789b0alc2931d27ef835d0eb71c8b95d77235248
    Image: nginx
    Image ID: docker.io/library/nginx@sha256:abe47724e466aeb9a345d8e46a221c2fa8953c7848bb4a3bd9976a7199f8cf2
    Port: 80/TCP
    Host Port: 0/TCP
    State: Running
      Started: Thu, 01 Oct 2026 08:17:14 +0000
    Ready: True
    Restart Count: 0
    Liveness: http-get http://:80/ delay=10s timeout=1s period=5s #success=1 #failure=3
    Readiness: http-get http://:80/ delay=5s timeout=1s period=5s #success=1 #failure=3
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-2jhg9 (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized         True
  Ready              True
  ContainersReady     True
  PodScheduled        True
Volumes:
  kube-api-access-2jhg9:
    Type: Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional: false
    DownwardAPI: true
QoS Class: BestEffort
Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type      Reason      Age   From          Message
  ----      -
  Normal    Scheduled   118s  default-scheduler  Successfully assigned default/probe-pod to k8s-worker2
  Normal    Pulling     117s  kubelet        Pulling image "nginx"
  Normal    Pulled      115s  kubelet        Successfully pulled image "nginx" in 2.235s (2.235s including waiting). Image size: 63422407 bytes.
  Normal    Created     115s  kubelet        Created container: nginx
  Normal    Started     115s  kubelet        Started container nginx
ubuntu@k8s-master:~$

```

Step 5.4 — Test readiness

First create a Service:

Nano probe-service.yaml

```

ubuntu@k8s-master: ~
GNU nano 8.7.1 probe-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: probe-service
spec:
  selector:
    app: probe-demo
  ports:
    - port: 80
      targetPort: 80

```

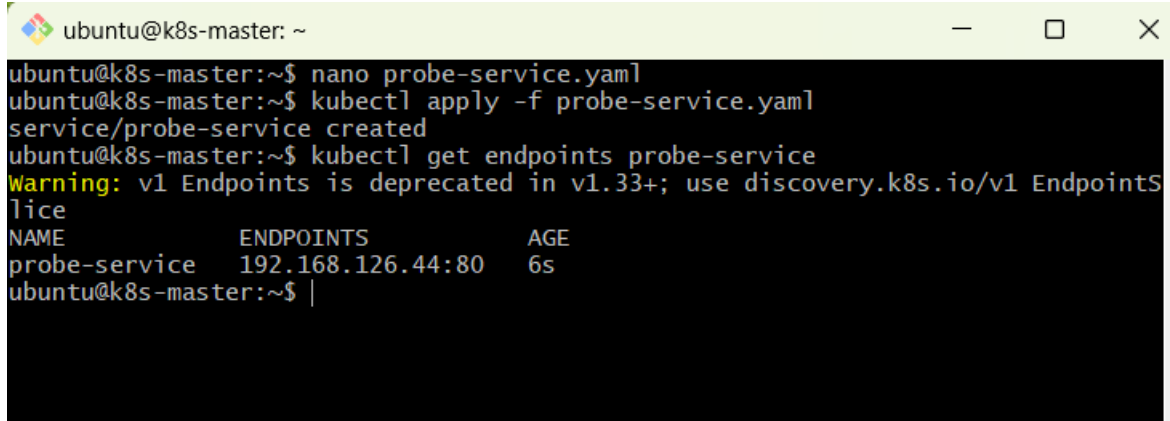
Apply:

kubectl apply -f probe-service.yaml

Check:

```
kubectl get endpoints probe-service
```

You should see the Pod IP.

A terminal window titled 'ubuntu@k8s-master: ~' with standard window controls. The terminal shows the following commands and output:

```
ubuntu@k8s-master:~$ nano probe-service.yaml
ubuntu@k8s-master:~$ kubectl apply -f probe-service.yaml
service/probe-service created
ubuntu@k8s-master:~$ kubectl get endpoints probe-service
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 Endpoints
lice
NAME           ENDPOINTS           AGE
probe-service  192.168.126.44:80   6s
ubuntu@k8s-master:~$ |
```

Step 5.5 — Break readiness

Now we'll make nginx unavailable.

Enter the Pod:

```
kubectl exec -it probe-pod -- bash
```

Stop nginx:

```
nginx -s stop
```

Exit:

```
exit
```

Check:

```
kubectl get pod probe-pod
```

You may see:

```
0/1
```

Now:

```
kubectl get endpoints probe-service
```

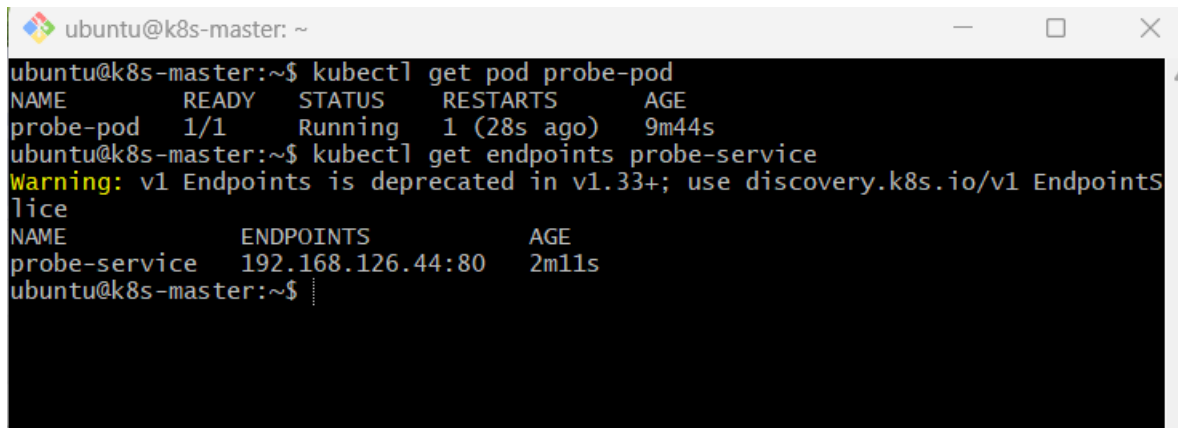
The endpoint should disappear while readiness is failing.

Important

Readiness failure does **not necessarily restart the container**.

It tells Kubernetes:

"Don't send traffic to me."

A terminal window titled 'ubuntu@k8s-master: ~' with standard window controls. It shows two kubectl commands and their outputs. The first command is 'kubectl get pod probe-pod', which returns a table with columns NAME, READY, STATUS, RESTARTS, and AGE. The second command is 'kubectl get endpoints probe-service', which returns a warning about deprecated v1 Endpoints and a table with columns NAME, ENDPOINTS, and AGE.

```
ubuntu@k8s-master:~$ kubectl get pod probe-pod
NAME      READY   STATUS    RESTARTS   AGE
probe-pod  1/1     Running   1 (28s ago) 9m44s
ubuntu@k8s-master:~$ kubectl get endpoints probe-service
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 Endpointslice
NAME      ENDPOINTS             AGE
probe-service 192.168.126.44:80 2m11s
ubuntu@k8s-master:~$
```

Step 5.6 — Liveness failure

The liveness probe will detect that nginx is no longer responding.

Check:

`kubectl describe pod probe-pod`

Look under:

Events:

You may see messages related to:

Liveness probe failed

Eventually Kubernetes restarts the container.

Check:

`kubectl get pod probe-pod`

And:

`kubectl get pod probe-pod -o wide`

Check restart count:

`kubectl get pod probe-pod`

Example:

NAME	READY	STATUS	RESTARTS
probe-pod	1/1	Running	1

```
ubuntu@k8s-master: ~  
Last State: Terminated  
Reason: Completed  
Exit Code: 0  
Started: Thu, 01 Oct 2026 08:17:14 +0000  
Finished: Thu, 01 Oct 2026 08:26:27 +0000  
Ready: True  
Restart Count: 1  
Liveness: http-get http://:80/ delay=10s timeout=1s period=5s #success=1 #failure=3  
Readiness: http-get http://:80/ delay=5s timeout=1s period=5s #success=1 #failure=3  
Environment: <none>  
Mounts:  
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-2jhgn (ro)  
Conditions:  
  Type                               Status  
  PodReadyToStartContainers          True  
  Initialized                         True  
  Ready                              True  
  ContainersReady                    True  
  PodScheduled                       True  
Volumes:  
  kube-api-access-2jhgn:  
    Type: Projected (a volume that contains injected data from multiple sources)  
    TokenExpirationSeconds: 3607  
    ConfigMapName: kube-root-ca.crt  
    Optional: false  
    DownwardAPI: true  
QoS Class: BestEffort  
Node-Selectors: <none>  
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300s  
             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s  
Events:  
  Type      Reason      Age           From          Message  
  ----      -  
  Normal    Scheduled   11m           default-scheduler Successfully assigned default/probe-pod to k8s-worker2  
  Normal    Pulled      11m           kubelet       Successfully pulled image "nginx" in 2.235s (2.235s including waiting). Image size: 63422407 bytes.  
  Normal    Pulling     107s (x2 over 11m) kubelet       Pulling image "nginx"  
  Normal    Created     107s (x2 over 11m) kubelet       Created container: nginx  
  Normal    Started     107s (x2 over 11m) kubelet       Started container nginx  
  Normal    Pulled      107s          kubelet       Successfully pulled image "nginx" in 129ms (129ms including waiting). Image size: 63422407 bytes.  
ubuntu@k8s-master:~$ kubectl get pod probe-pod  
NAME      READY   STATUS    RESTARTS   AGE  
probe-pod 1/1     Running   1 (117s ago) 11m  
ubuntu@k8s-master:~$ kubectl get pod probe-pod -o wide  
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES  
probe-pod 1/1     Running   1 (2m9s ago) 11m   192.168.126.44 k8s-worker2 <none>          <none>  
ubuntu@k8s-master:~$
```

Very Important Difference

Remember this table:

Probe	Question	Failure
Liveness	Is application alive?	Container may restart
Readiness	Can application receive traffic?	Removed from Service endpoints
Startup	Has application finished starting?	Protects slow-starting applications

Think about a real application: